

CyberGuard: A Simple Guide

This guide explains how the CyberGuard project works in simple, non-technical terms. Read this to easily understand the project and confidently explain it to the jury.

1. What is CyberGuard?

CyberGuard is a security application that acts like a digital bodyguard. When you receive a suspicious link or text message, you paste it into CyberGuard. It instantly tells you if the link is a dangerous phishing attempt, or if it is safe to click.

2. The Two Halves of the Project

Think of the project like a restaurant:

- **The Frontend (The Dining Area):** This is the part the user sees on their screen. We built this using 'Next.js' and 'React'. It provides the beautiful design, the input boxes, and the red/green warning dials.
- **The Backend (The Kitchen):** This is the hidden engine that does all the hard work. We built this using 'Python' and 'FastAPI'. When the user pastes a link on the screen, the frontend sends it to the backend kitchen to be inspected.

3. How does it spot a fake link?

When the backend receives a link, it doesn't just guess. It passes the link through two layers of security:

Layer A: The Rule-Checker (Heuristics)

First, the program checks the link against a strict set of human rules. It looks for obvious red flags:

- *Typosquatting*: Is it trying to trick you by spelling a brand wrong? (e.g., 'net-flix.com' instead of 'netflix.com')
- *Hidden Subdomains*: Is the real website hiding? (e.g., 'paypal.com.scam-site.com')
- *Direct Downloads*: Will clicking the link instantly force you to download an .exe virus?

Layer B: The Artificial Intelligence (Machine Learning)

If the link passes the basic rules, we hand it over to the AI (Artificial Intelligence).

- We trained our AI model on over 800,000 real-world examples of safe and malicious links.
- The AI turns the words in the URL into mathematics (a process called 'TF-IDF').
- The AI then acts like a detective, recognizing hidden mathematical patterns that humans can't see, and gives the link a 'Threat Score' out of 100.

4. The Failsafe (Whitelisting)

Sometimes, Artificial Intelligence gets confused and flags a safe website as dangerous. To prevent this, we added a 'Whitelist Override'. If our Rule-Checker sees that a link is exactly 'paypal.com' or 'github.com', it immediately overrides the AI and forces the threat score to 0 (Safe).

5. The Final Output

Once the Python backend finishes checking the rules and asking the AI, it gathers all its findings and sends a report back to the frontend screen.

The user doesn't see the complicated math. They just see a clean dashboard telling them exactly WHY the link is dangerous (e.g., 'Warning: This link is hiding its true destination.').

6. Key Takeaways for the Jury

- **Hybrid System:** Emphasize that you didn't just use AI. You used AI combined with strict rules, which is how professional cybersecurity companies build tools.
- **Explainability:** Emphasize that your tool doesn't just say 'Bad Link'. It explains exactly WHY the link is bad, which helps educate users.
- **Full-Stack:** You built a complete product. A modern React user interface connected to a high-speed Python AI engine.